

GIT Commands

Class no:3

Git Merge

Git Merge combines changes from different branches into a single branch, integrating work while preserving history. It helps unify development without losing progress.

- Preserves commit history and creates a merge commit when needed.
- Performs fast-forward or automatic merges unless conflicts occur.
- Commonly used to integrate feature branches into the main branch.

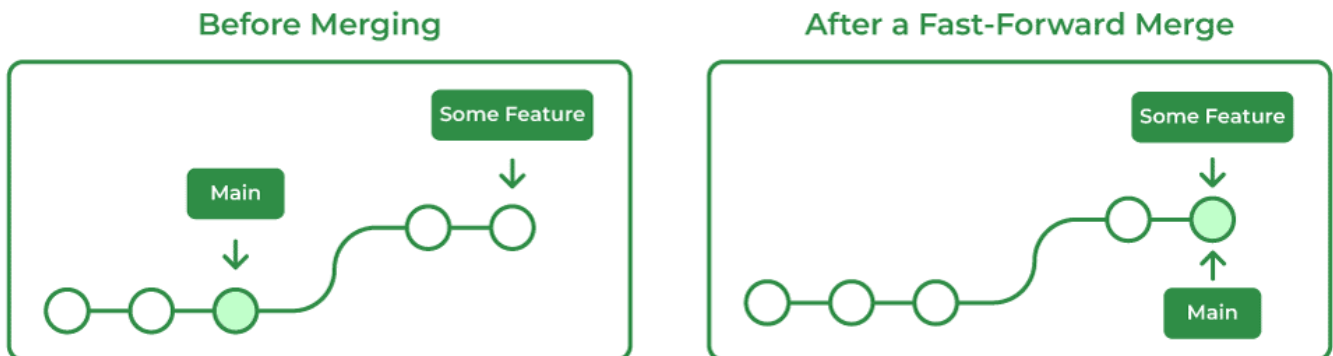
Types of Merging in Git

Git supports several types of merging. The two most common types are:

1. Fast-forward merging

Occurs when Git advances the current branch pointer to the latest commit without creating a merge commit.

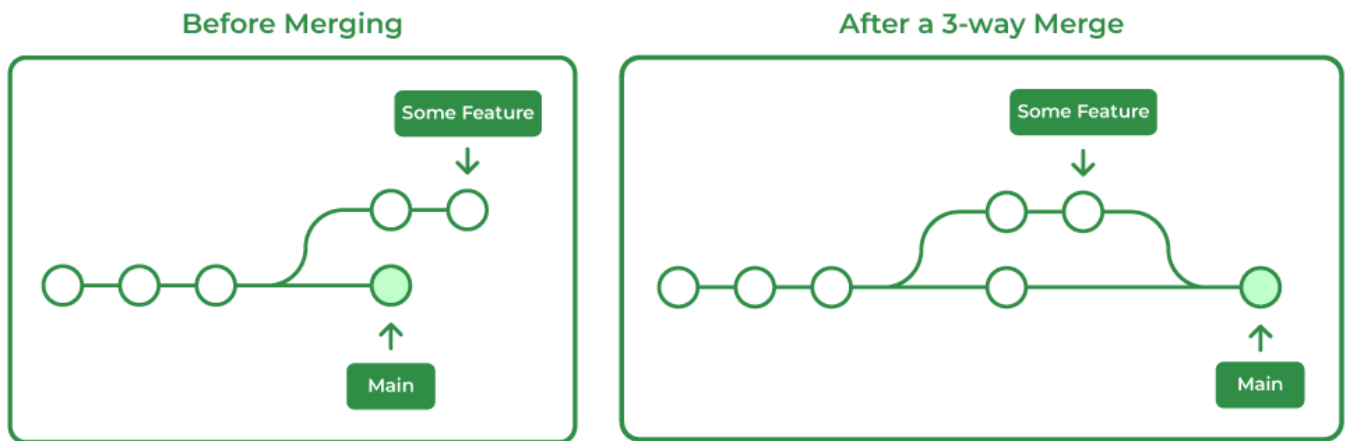
- Happens when the target branch is directly ahead of the current branch.
- Git moves the branch pointer forward instead of creating a new commit.
- Fast-forward merging only possible when branches have not diverged.



2. Three-way merging

Occurs when branches have diverged, so Git creates a new merge commit by combining their changes.

- Happens when the base branch has new commits after branching.
- Git compares both branches with a common base commit.
- A new merge commit is created to combine changes.



Git - Merge

Note: Git also supports recursive and octopus merging, where recursive is the default strategy for complex histories and octopus merge combines multiple branches in a single commit.

What is a Pull Request?

A **pull request (PR)** in GitHub is a collaborative tool used to propose and review changes to a codebase before merging them into the main branch. It allows developers to notify others about updates made in a feature branch and facilitates discussions, code reviews, and quality checks before integrating the changes into the target branch.

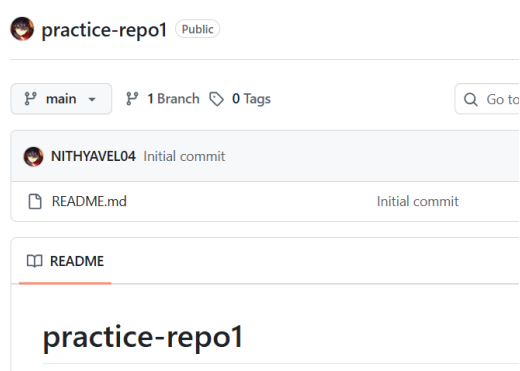
How Pull Requests Work

1. **Create a Feature Branch:** Developers create a new branch from the main branch to work on a specific feature or bug fix.
2. **Commit Changes:** Code changes are made and committed to the feature branch.
3. **Push to Remote Repository:** The feature branch is pushed to the remote repository on GitHub.
4. **Open a Pull Request:** A pull request is created to propose merging the feature branch into the target branch (e.g., main).
5. **Review and Feedback:** Team members review the changes, leave comments, and suggest improvements.
6. **Address Feedback:** The author updates the pull request with the requested changes.
7. **Merge the Pull Request:** Once approved, the pull request is merged into the target branch.

Hands on implementation of Merge & Pull requests

(Note : Both are same. Merge will be done in the CLI while Pull will be done in the GitHub console).

Step no:1 Create a remote repository in the git hub



Step no:2 Now open the git bash create a folder and clone the repository inside it.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit (main)
$ git clone https://github.com/NITHYAVEL04/practice-repo1.git
Cloning into 'practice-repo1'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

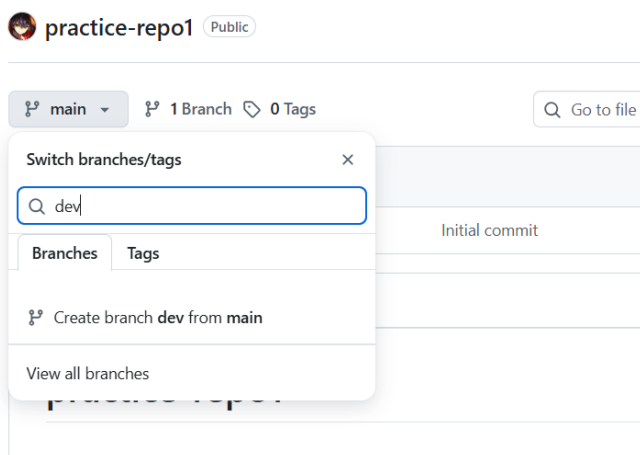
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit (main)
$ ls
practice-repo1/

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit (main)
$ cd practice-repo1/
```

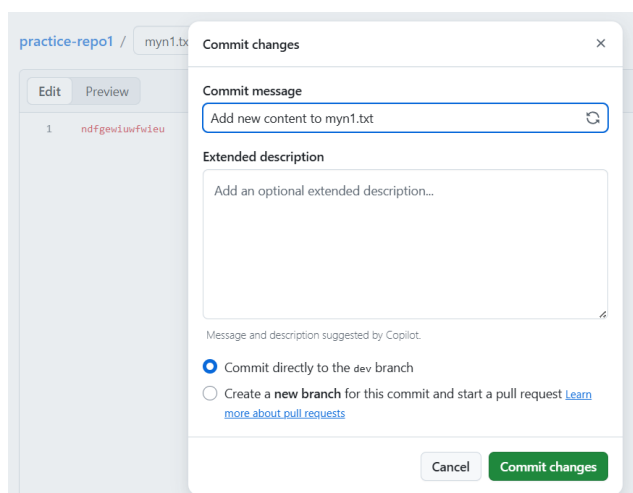
Step no:3 The git clone is used to bring the remote repo to the local, while the git pull is to bring the updates and changes made in the remote repo to the local.

Create a branch named: **dev** from **Main**.

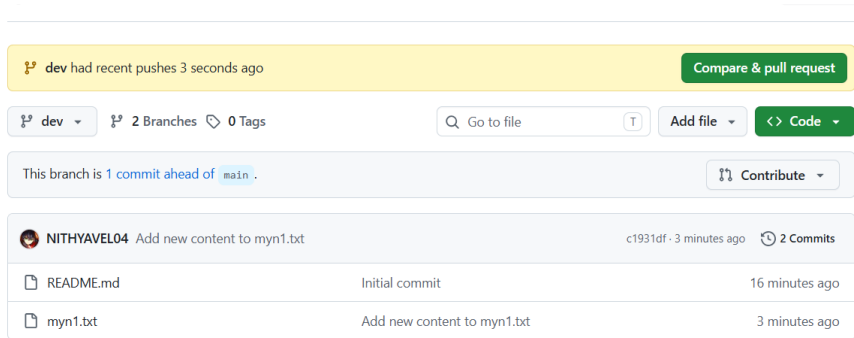
Be aware of where we are creating the branch.



Step no:4 Now open the **dev** branch and add a file in it. Commit changes.



Now we can see a pull request has been shown in the git hub console. If we accept that this branch will be merged with the Main branch. Currently, ignore that.



Step no:5 Now we are going to create a branch and merge it with main from cli.

So create a branch inside that repo from cli with same name of the branch in the git Hub.

Then switch to the branch

Once switched, pull the branch

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit (main)
$ cd practice-repo1/

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ ls
README.md

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ git branch dev

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ git switch dev
Switched to branch 'dev'

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (dev)
$ git pull origin dev
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 962 bytes | 64.00 KiB/s, done.
From https://github.com/NITHYAVEL04/practice-repo1
* branch                dev                -> FETCH_HEAD
* [new branch]          dev                -> origin/dev
Updating 6c47028..c1931df
Fast-forward
 myn1.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 myn1.txt

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (dev)
$ |
```

We can see the file that we have added in the remote repo dev branch in the local repo dev branch and shows both the commits of main as well as the dev.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (dev)
$ ls -a
./ ../ .git/ README.md myn1.txt

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (dev)
$ git log --oneline
c1931df (HEAD -> dev, origin/dev) Add new content to myn1.txt
6c47028 (origin/main, origin/HEAD, main) Initial commit

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (dev)
$ |
```

Step no:6 In real life scenarios, we will perform the updates and changes in the respective branch (here dev). Once the work has been completed, it will be merged with the main branch with approvals from the collaborators (delegates and colleagues).

To perform that,

First move to the main branch. (The branch must be where we are going to merge our changes, here I am using main)

Now merge our branch (dev) with the main.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (dev)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ git merge dev
Updating 6c47028..c1931df
Fast-forward
 myn1.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 myn1.txt

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ |
```

We can see both the commits in commit history of main as it is merged with dev locally.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ git log --oneline
c1931df (HEAD -> main, origin/dev, dev) Add new content to myn1.txt
6c47028 (origin/main, origin/HEAD) Initial commit

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$
```

In order to see the changes in the remote repo as well we need to push it or else the changes will be only in the local.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/NITHYAVEL04/practice-repo1.git
 6c47028..c1931df  main -> main

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/practicegit/practice-repo1 (main)
$ |
```

practice-repo1 Public

main 2 Branches 0 Tags

Go to file

Add file Code

NITHYAVEL04 Add new content to myn1.txt c1931df · 21 minutes ago 2 Commits

README.md	Initial commit	34 minutes ago
myn1.txt	Add new content to myn1.txt	21 minutes ago

README

practice-repo1

Step no:7 Now we are going to create a pull request in the git Hub.

Create a new branch from dev and add a file. Commit changes.

practice-repo1 Public

dev 2 Branches 0 Tags

Go to file

Switch branches/tags

test

Branches Tags

Create branch test from dev

View all branches

Initial commit

Add new content to myn1.txt

< Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Files

test

Go to file

README.md

myn1.txt

practice-repo1 / mytest.py

Edit Preview

```
1 print("hello")
```

Commit changes

Commit message

Add print statement to mytest.py

Extended description

Add an optional extended description...

Message and description suggested by Copilot.

☒ Commit directly to the test branch

☐ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)

Cancel Commit changes

Now the test branch will have three commits.

 test  3 Branches  0 Tags

Go to file  Add file  Code

This branch is 1 commit ahead of main  Contribute

 NITHYAVEL04 Add print statement to mytest.py b589a76 · now 3 Commits

 README.md	Initial commit	44 minutes ago
 myn1.txt	Add new content to myn1.txt	30 minutes ago
 mytest.py	Add print statement to mytest.py	now

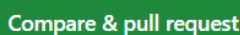
Step no:8 In pull request we will add a reviewer and merge the branches.

In merge in cli we just merge, we don't add any reviews.

Hence pull requests are permissions in higher level to merge the branches.

Issues  Pull requests  Agents  Actions  Projects  Wiki  Security and quality  Insights  Settings

 practice-repo1 Public  Pin  Watch

 test had recent pushes 2 seconds ago 

Either click on the new pull request notification or the pull request option in the tool bar.

I am going to create a new pull request, so I am clicking the one in the tool bar → **new pull request** → **base: dev** and **compare: test**

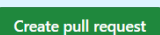
It will compare the changes. In case of change it will show. If no changes it will show that there is nothing to change.

Click create pull request.

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

 base: dev  compare: test  **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#) 

 1 commit  1 file changed  1 contributor

 Commits on May 14, 2026

 Add print statement to mytest.py   b589a76 

 NITHYAVEL04 authored 7 minutes ago

 Showing 1 changed file with 1 addition and 0 deletions.  Split  Unified

 1 mytest.py 

 ...  ...  @@ -0,0 +1 @@

 1 + print("hello")

It will ask for title and description provide it and create.

base: dev

compare: test

✓ Able to merge. These branches can be automatically merged.

Add a title *

Add print statement to mytest.py

Add a description

Write

Preview

H B I

dbiwqduq

Markdown is supported

Paste, drop, or click to add files

Create pull request

Step no:9 In pull request we can add a reviewer for best practice. We can add up to 15 reviewer.

Even though we can merge without a reviewer.

NITHYAVEL04 commented now

dbiwqduq

No conflicts with base branch
Merging can be performed automatically

Merge pull request

Step no:10 In order to add a reviewer → go to repository settings → collaborators

NITHYAVEL04 / practice-repo1

Issues Pull requests 1 Agents Actions Projects Wiki Security and quality Insights Settings

General

Access

Collaborators

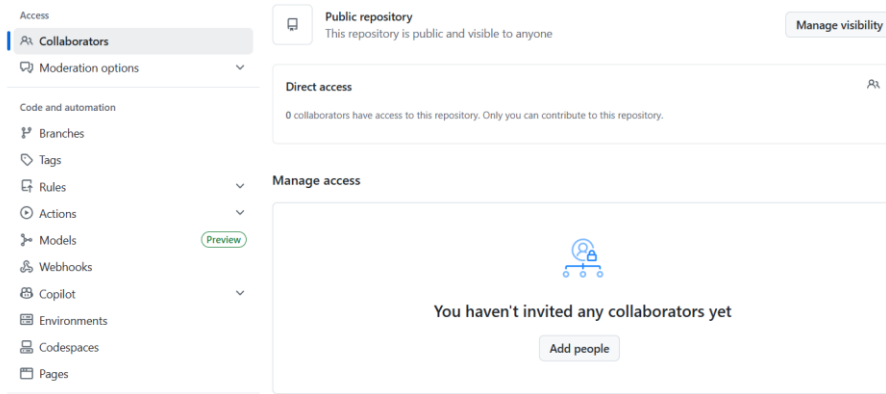
Moderation options

Repository name

practice-repo1

Rename

In manage access we can add people using their git Hub id



The respective collaborator has to accept the invitation.

Once added the pull request will be sent to them, they will check and provide corrections and approval.

Once approved we can merge the branches. It will create a new commit.

Git Remote -v command

git remote -v is your "eyes" on where your code is actually going when you push or pull. It is a simple but essential command for managing the connections between your local repository and the servers where your code is hosted (like GitHub, GitLab, or Bitbucket).

What does it do?

The -v in the command stands for **verbose**.

When you run git remote, it simply lists the short names of your remote connections (e.g., origin). When you add the -v flag, it reveals the specific **URLs** associated with those names and what they are used for.

Specifically, it shows you:

1. **The Remote Name:** Usually origin by default.
2. **The URL:** The web address or SSH path of the repository.
3. **The Purpose:** Whether the URL is used for **(fetch)** (pulling data) or **(push)** (sending data).

How to use it

Using the command is straightforward. Open your terminal or command prompt inside your project folder and type:

Bash

```
git remote -v
```

Example Output:

If you have a project cloned from GitHub, your output will look something like this:

origin <https://github.com/user/repository.git> (fetch)

origin <https://github.com/user/repository.git> (push)

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~  
$ cd D:  
  
nithya@DESKTOP-NBCS2S9 MINGW64 /d  
$ ls  
'$RECYCLE.BIN'/'CSA Review'/'Java_preparation/'  
'Bot based system aws implementation.docx' CUREX_APP.zip 'My personal downloads'/  
  
nithya@DESKTOP-NBCS2S9 MINGW64 /d  
$ cd 'My personal downloads'/  
  
nithya@DESKTOP-NBCS2S9 MINGW64 /d/My personal downloads  
$ ls  
'AWS introduction and EC2 cretion in linux and windows.pdf' 'Git practice'/ Linux  
  
nithya@DESKTOP-NBCS2S9 MINGW64 /d/My personal downloads  
$ cd 'Git practice'/  
  
nithya@DESKTOP-NBCS2S9 MINGW64 /d/My personal downloads/Git practice (main)  
$ ls  
'1_Git and Git hub Introduction.pdf' '2_Git Branch Overview.pdf' '3_GIT Commands.d  
  
nithya@DESKTOP-NBCS2S9 MINGW64 /d/My personal downloads/Git practice (main)  
$ git remote -v  
origin https://github.com/NITHYAVEL04/Git-practice.git (fetch)  
origin https://github.com/NITHYAVEL04/Git-practice.git (push)  
  
nithya@DESKTOP-NBCS2S9 MINGW64 /d/My personal downloads/Git practice (main)  
$ |
```